

Module 11: Inheritance

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

Inheritance is introduced as a direct answer to a concrete pain point: several related classes that would otherwise duplicate the same fields and methods. Anchor the whole session in the 'is-a' test and one running example (BankAccount / SavingsAccount) before generalizing to the track-specific hierarchies students will build in their own project.

Suggested Duration: 75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Apply the 'is-a' test to decide when inheritance is the right relationship.
- Write a derived class that inherits from a base class and adds new members.
- Use protected appropriately, distinguishing it from private and public.
- Override a base class method and call it correctly through a derived object.
- Explain construction order (base-first, then derived) and write a correct member initializer list.

Prerequisites

- Module 8 (Classes and Objects) from Part 1 - inheritance builds directly on class design fundamentals.

Materials & Instructor Prep

- 11-inheritance/examples/01_basic_inheritance.cpp preloaded, ready to extend live into a second derived class.

Key Vocabulary

Term	Definition
Base class (superclass/parent)	The class being inherited from.

Term	Definition
Derived class (subclass/child)	The class that inherits from a base class.
"is-a" relationship	The test for whether inheritance is appropriate: 'is a SavingsAccount a kind of BankAccount?'
protected	An access level between private and public: visible inside the class AND any class that derives from it, but not from outside code.
Member initializer list	The <code>: BaseClass(args)</code> syntax in a derived constructor's header that constructs the base part of the object first.
Overriding	A derived class redefining a method it inherited, changing its behavior for objects of that derived type.

Lesson Outline & Instructor Talking Points

1. The 'is-a' test, with a running example (15 min)

- Ask: 'is a SavingsAccount a kind of BankAccount?' Yes. Contrast with a clearly wrong pairing ('is a BankAccount a kind of Customer?' No - that's a 'has-a' relationship, not covered here) so the test has real discriminating power, not just a yes-by-default feel.
- Live-build the BankAccount base class first, exactly as in Module 8, before touching SavingsAccount at all - let students see it's the SAME class they already know how to write.

2. Deriving SavingsAccount (20 min)

- Write `class SavingsAccount : public BankAccount` live. Pause immediately on the colon syntax and name it explicitly.
- Show that `deposit()/getBalance()` work on a SavingsAccount object with ZERO new code - this is the payoff, make it visible and celebrated, not rushed past.
- Introduce the member initializer list only once it's actually needed (adding the constructor) - motivate it as 'BankAccount doesn't have a no-argument constructor, so we must tell it how to construct itself.'

3. protected - why a third access level (15 min)

- Recall private/public from Module 8. Ask: 'if balance is private, can SavingsAccount's `applyInterest()` reach it?' Let them predict no, then show the compiler error live.
- Introduce protected as the fix, and state the rule precisely: accessible inside the class AND its derived classes, still not from outside code.

4. Overriding a method (15 min)

- Switch to the Component/Resistor example (examples/02_overriding.cpp) - a cleaner illustration since the override changes OUTPUT, not just internal state.
- Show describe() called on a Resistor variable directly runs Resistor's version - but flag explicitly, as a cliffhanger: 'this only works reliably when you call it through a Resistor variable directly. Next module breaks this on purpose to show you why.'

Instructor note: *This cliffhanger framing is deliberate and important - it sets up Module 12's entire motivation. Don't resolve the tension here; let it sit.*

5. Construction order (10 min)

- State plainly: base-first, then derived, always. Trace through SavingsAccount's construction step by step on the board.

Common Student Misconceptions

Misconception	How to Address It
"A derived class automatically gets the base class's constructors as its own."	It doesn't - you write the derived constructor yourself and explicitly call a base constructor from its member initializer list.
"private and protected are basically the same thing."	private blocks even derived classes; protected explicitly allows derived classes through while still blocking outside code. Demonstrate the compiler error for the private case live.
"Overriding a method changes it everywhere, for every object of the base type too."	It only changes behavior for objects of the DERIVED type - a Component variable still uses Component's own describe(), never Resistor's, unless something more (Module 12) is involved.

Exercises & Module Project

Exercise: 11-inheritance/exercises/exercise1.cpp - a Vehicle/ElectricVehicle hierarchy. Project (three tracks): Generic - Shape hierarchy; EE - Circuit Component hierarchy (Resistor/Capacitor/Inductor); Biomedical - Medical Device hierarchy (Thermometer/PulseOximeter). All three build a base class plus 2-3 derived classes, each adding its own fields and overriding a shared method.

Facilitation guidance: Whichever track a student picks, check specifically for: correct use of protected (not private) for base members the derived class touches directly, a correct member initializer list in every derived constructor, and a genuinely overridden (not just duplicated) shared method.

Discussion & Comprehension Check Questions

1. How do you decide whether two classes should be related by inheritance?

Guidance: Apply the 'is-a' test - if it doesn't read naturally as 'X is a kind of Y,' inheritance is probably the wrong tool.

2. Why does a derived class's constructor need to call the base class's constructor explicitly?

Guidance: The base part of the object must be fully constructed before the derived part can safely add anything on top of it - there's no way to skip this.

Assessment Ideas

- Design check: give students a short scenario (e.g. 'a Car and a Truck both have a make and top speed, but a Truck also has a cargo capacity') and have them sketch just the class headers and member initializer lists, no full implementation.

Answer Key Notes

See [11-inheritance/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Remember the cliffhanger from the overriding example? Next module resolves it - what happens when you try to treat a Resistor through a generic Component pointer, which is exactly what you need for a mixed collection of components."

Instructor Tips

- Keep the SAME base example (BankAccount/Component, whichever fits your group) running across this module and the next - reusing it makes Module 12's payoff land harder, because students already trust the setup.